PHP/MySQL

ESGIS Cotonou

Dr. Béthel ATOHOUN
Directeur des Etudes de l'ESGIS Bénin

Mai 2023

OBJECTIFS DU COURS

L'objectif de ce cours est de donner à l'apprenant les notions de base de PHP/MySQL afin qu'il puisse exploiter au mieux les capacités de PHP dans le cadre du développement Web dynamique

À la fin de ce cours, l'étudiant sera en mesure de :

- ❑ Générer dynamiquement des pages web
- ❑ Contrôler l'affichage des contenus d'une page web
- ❑ Interagir avec une base de données pour y Ajouter/Supprimer/Mettre à jour des données depuis un formulaire
- ❑ Authentifier un utilisateur et lui associer une session

DÉFINITION ET PRÉREQUIS

DÉFINITION ET PRÉREQUIS

Définitions

PHP = *PHP Hypertext PreProcessor*

*PHP = Personal Home Page (au départ par **Rasmus Lerdorf** le créateur)*

Catégorie de langage

PHP est un langage de script (langage de programme) coté serveur, utilisé en Développement Web Dynamique.

PHP est un langage interprété

PHP est sensible à la case (donc fait la différence entre minuscule et majuscule

PHP est un langage de programmation à part entière dont les domaines de compétence vont au-delà du développement Web. Il permet de générer des images, des fichiers pdf, etc. Il peut s'utiliser de façon couplée avec un serveur web, comme il peut aussi s'utiliser de façon autonome en ligne de commande. Il dispose d'extension permettant la création d'application graphiques standards de type desktop.

DÉFINITION ET PRÉREQUIS

Atouts

- Très forte capacité d'intégration d'applications et de sources de données. (Champion reconnu de l'intégration bas niveau)
- *Parfaite intégration avec le HTML et le XML*
- *Possibilité de connexion à la plus part des bases de données*
- *Support de la quasi-totalité des protocoles et formats relatifs à l'Internet*
- *Très stable, très fiable et assez performant*
- *Le code PHP est totalement portable, donc multi plates-formes*

PHP est dès la base Open Source, donc totalement gratuit. Son code source est disponible en libre téléchargement. PHP dispose d'une très grande communauté d'utilisateurs. Cette communauté est très organisée et très réactive.

DÉFINITION ET PRÉREQUIS

Prérequis

PHP permet de faire beaucoup de choses mais sa facette qui nous intéresse ici est celle permettant la gestion dynamique de contenu pour le Web. Dans ce cadre, il nous faudra :

- *Un serveur web (ici Apache)*
- *Un moteur d'interprétation de code PHP (Zend engine)*
- *Un SGBD (ici le MySQL qui est natif à PHP mais on peut utiliser un autre)*
- *Un éditeur de code (Notepad++ ou un autre)*

Pour faire du développement Web statique ou dynamique on aura besoin d'avoir tout au moins des prérequis en HTML (aujourd'hui HTML5)

DÉFINITION ET PRÉREQUIS

Solutions tout-en-un pour programmer en PHP

Plusieurs approches existent aussi bien sous Windows que sous Linux pour installer et configurer un serveur web, un gestionnaire de bases de données et PHP. Mais il existe des solutions tout-en-un très pratiques et très facile à mettre en œuvre.

Sous Windows, on peut distinguer WAMP, XAMPP, EasyPHP pour ne citer que celles là. Sous Linux on a XAMPP.

– *L'installation est facile et intuitive.*

Pour faire ce cours nous allons nous baser sur WAMP (Windows-Apache-MySQL-PHP).

PREMIÈRE PARTIE

LES ÉLÉMENTS DE BASE

Insertion de PHP dans du code HTML

Ouverture	Fermeture
<?php	?>
<?	?>
<%	%>
<script language= "php">	</script>

Commentaires

/* commentaire de plusieurs lignes */

//commentaire d'une ligne ou fi n d'une ligne

Pour des raisons de portabilité, il est fortement recommandé de n'utiliser que la balise d'ouverture **<?php** sinon il faudra s'assurer de l'activation ou non de certaines options dans les fichiers de configuration.

LES ÉLÉMENTS DE BASE

Variables et constantes

Variable : Le nom d'une variable PHP est composée du caractère **$** suivi du nom de la variable (qui est un identificateur). La syntaxe est la suivante :

$nom_variable

- Portée d'une variable (espace du script dans lequel elle va être accessible)
- Variable locale , variable globale (mot clé **global** et tableau associatif **$GLOBALS**)
- Fonctions **isset** , **empty** et **unset**

isset permet de vérifier l'existence d'une variable. **empty** permet de voir si le contenu d'une variable (qui existe) est vide ou non. **unset** permet de détruire une variable.

En PHP, l'utilisation des variables est très simple et ne nécessite aucune déclaration préalable. Le type d'une variable est déterminé dynamiquement par le type de son contenu à l'instant. On n'a donc pas besoin de déclarer une variable avant de l'utiliser.

LES ÉLÉMENTS DE BASE

Variables et constantes

Exemples :

```
<?php
    $nom ="Moussa";
    $age=12;
    if(isset($nom)&& !empty($nom))
    {
        echo "Le contenu de nom est :"  .$nom;
        unset($nom);
    }
?>
```

LES ÉLÉMENTS DE BASE

Variables et constantes

Constante : La définition des constantes se fait à l'aide de la fonction **define().**

La syntaxe est la suivante : `define('NOM_CONSTANTE', "valeur") ;`

Exemple :

```
<?php
  define('MESSAGE', "Bienvenue à la formation PHP Débutant");
  echo  MESSAGE;
?>
```

Il faut noter la nom présence du $ au début du nom d'une constante. Il faut dire que par convention, le nom d'une constante est en majuscule.

LES ÉLÉMENTS DE BASE

TYPE DE DONNEES

PHP supporte 8 types de base, à savoir :

4 types scalaires : booléen, Entier, Réel (nombre à virgule flottante), chaine de caractères

2 types structurés : tableau et objets

2 types spéciaux : ressource et NULL

Ci-dessous, quelques fonction de gestion des types

Quelques fonctions

- `gettype()`
- `is_string()`
- `is_double()`
- `is_float()`
- `is_int()`
- `is_integer()`
- `is_boolean()`
- `is_array()`
- `is_null()`
- `is_object()`
- `is_resource()`
- `settype()`

LES ÉLÉMENTS DE BASE

TYPE DE DONNEES

Les tableaux : En PHP il y a deux types de tableaux.

- Tableau indexé numériquement (indice numérique)
- Tableau associatif (indice sous forme de texte)

On n'a pas besoin de préciser la taille des tableaux lors de la déclaration.

Création : Il existe plusieurs possibilités qui se basent sur 2 approches de base

- Ecriture directe dans les cellules
- Utiliser le mot clé **array**

Depuis la version 5.4 de PHP, on peut aussi utiliser une syntaxe inspirée du langage python pour créer un tableau.

LES ÉLÉMENTS DE BASE

TYPE DE DONNEES

Les tableaux

Exemples : Ci-dessous 4 exemples de création de tableaux.

```
$Tab1 = array(123,  "texte 1", "texte 2" , 456, 14);

$ Tab2= array("nom " => "MOUSSA ", "age "=>21, "taille"=>1.80);


$ Tab3[0]="Bonjour";
$Tab3[1]=45;
 $Tab3[ ]= "Merci ";
$Tab3 [ ]=12.5;

$Tab4 = [
   “ecole" => “ESGIS",
   “contact" => “21324773",
];
```

LES ÉLÉMENTS DE BASE

Les Opérateurs

Opérateur d'affectation

Opérateur	Exemples	Commentaire
=	$x=4; $val="message"	Possibilité d'affectation multiple

Opérateur d'affectation par copie et références

Opérateur	Exemples	Commentaire
= &	$x=4; $y=& $x; $y=$y+1;	La nouvelle valeur de $y sera 5 et il en sera de même pour $x

Opérateurs incrémentation

Opérateur	Opération	Exemple
++	incrémentation	$x=5; $x++; //$x contient finalement 6
--	décrémentation	$y=5; $y--; //$y contient finalement 4

LES ÉLÉMENTS DE BASE

Opérateurs arithmétiques

Opérateur	opération	Exemple
+	Addition	$c = $a + $b;
-	Soustraction	$c = $a - $b;
*	Multiplication	$c = $a * $b;
/	Division (y compris entière)	$c = $a / $b;
%	Modulo	$c = $a % $b;

Opérateurs combinés (ou associatifs)

opérateur	opération
+=	Ajoute la valeur de droite à la valeur de gauche.
-=	Ajoute la valeur de droite à la valeur de gauche.
*=	Multiplie la valeur de droite par la valeur de gauche.
etc.	/=, &=, %= , |=, .=

LES ÉLÉMENTS DE BASE

Opérateur de concatenation

Opérateur	opération	Exemple
.	concaténation	$a="Un "; $b=" message"; $c= $a .$b ."concaténé";

Opérateurs de comparaison

Opérateur	Opération	Exemple
==	Egal à	$x==$y
===	Identique à	$x===$y //égal en valeur et en type
<	Inférieur à	$x<$y
>	Supérieur	$x>$y
<=	Inférieur ou égal à	$x<=$y
>=	Supérieur ou égal à	$x>=$y
!=	Différent de	$x!=$y
!==	Pas identique à	$x!==$y // différent en valeur ou en type

LES ÉLÉMENTS DE BASE

Opérateurs logiques

Opérateur	Opération	Exemple
!	Négation (NON)	! $a
&&	ET	$a && $b
||	OU	$a || $b
and	ET	Il faut parenthèser pour éviter les problèmes
or	OU	Il faut parenthèser pour éviter les problèmes
xor	OU exclusif	$a xor $b

LES ÉLÉMENTS DE BASE

Les Structures de contrôle

Les structures conditionnelles

- **if**

Syntaxe :

```
if (<condition>)
{
instruction(s);
}
```

if … else

Syntaxe

```
if (<condition>){
        instruction(s);}
else {
        instruction(s);
}
```

LES ÉLÉMENTS DE BASE

Les structures conditionnelles

if … elseif … else

Syntaxe

```
if (<condition1>){
        instruction(s);
}
elseif (<condition2>) {
        instruction(s);
}
elseif (<condition3>) {
        instruction(s);
}
.
.
.
elseif (<condition3>) {
        instruction(s);
}
else {
        instruction(s);
}
```

LES ÉLÉMENTS DE BASE

Les structures conditionnelles

- **switch**

```
switch(expression){
        case valeur1 : instruction(s);
                       break;
        case valeur2 : instruction(s);
                       break;
        .
        .
        .
        case valeurN : instruction(s);
                       break;

        default : instruction(s);
}
```

LES ÉLÉMENTS DE BASE

- **Opérateur conditionnel** ( ) ? :

Les structures itératives

- **for**

```
for (initialisation ; condition ; incrément) {
  instruction(s);
}
```

- **while**

```
while ( condition ){
  instruction(s);
}
```

LES ÉLÉMENTS DE BASE

Les structures itératives

- **do … while**

```
do{
  instruction(s);
} while ( condition )
```

- **foreach** (pour les tableaux et les objets)

```
foreach ($table as $element)
{
   instruction(s);
}
foreach ($table as $cle=>$element)
{
   instruction(s);
}
```

LES ÉLÉMENTS DE BASE

Instructions de rupture de contrôle

- **break**,

L'instruction **break** permet de sortir d'une structure conditionnelle telle que for, while, foreach ou switch.

- **continue**,

L'instruction **continue** est utilisée dans une boucle afin d'éluder les instructions de l'itération courante et donc pour passer directement à la suivante.

- **return**

L'instruction **return** est utilisée à l'intérieur d'une fonction pour mettre fin à son exécution et retourner la main au programme appelant en lui mettant éventuellement à disposition une valeur de retour.

LES ÉLÉMENTS DE BASE

Les Fonctions

Il existe deux types de fonctions en PHP : les fonctions natives, utilisables sans faire appel à des bibliothèques, et les fonctions utilisateurs, qui sont déclarées par vous-même ou qui sont définies dans une bibliothèque.

Déclaration d'une fonction

On utilise le mot-clé **function**.

La syntaxe est la suivante :

```
function Nom_de_La_Fonction([$argument1, $argument2, ...]) {
// Liste d'instructions ;
[return valeur;]
}

function Nom_de_La_Fonction([$argument1, $argument2='valeur']) {
// Liste d'instructions ;
[return valeur;]
}
```

LES ÉLÉMENTS DE BASE

Les Fonctions

Visibilité des variables

Les variables déclarées dans une fonction ne seront utilisables que dans celle-ci.

Les variables déclarées dans le script ne seront pas accessibles dans une fonction.

Les deux espaces sont complètements indépendants.

Portée des variables

- variable locale
- variable globale
- variable statique

Nota : pour accéder dans une fonction à une variable globale on utilise le mot clé **global**. Une variable statique est une variable locale persistante. On la crée à l'intérieur de la fonction avec le mot clé **static**.

LES ÉLÉMENTS DE BASE

Les Fonctions

Exemple

```
$message = 'Nombre de passage : ';

function compteur($mode='') {
        global $message;
        static $nb=0;
        $nb++;
        // On incrémente le nombre de camions
        if($mode == 'affiche') {
        echo $message . $nb;

       }

}
```

Nota : Sauf précision, tout paramètre passé à une fonction, est passée par valeur (donc par copie). La seule exception à cette règle concerne les objets, qui sont toujours passés par référence depuis PHP 5.

GÉNÉRALITÉS : INCLUSION DE FICHIERS

- **include** et **require**
  - Permettent d'inclure le contenu du fichier spécifié en argument
  - Différentes au niveau de la gestion des erreurs
  - **include** peut-être conditionnel alors que **require** ne peut l'être.

- **include_once** et **require_once** => éviter des inclusions multiples.

include émettra un message d'**alerte** (warning) si elle ne trouve pas un fichier spécifié tandis que **require** émettra une erreur fatale dans le même cas. Autrement, **require** est bloquant tandis que **include** ne l'est pas.

GÉNÉRALITÉS : LES COOKIES

- **Cookies**

- Fichiers texte courts stockés par le navigateur sur l'ordinateur de l'utilisateur, à la demande du serveur web.
- Suites d'informations transmises en en-tête HTTP par le serveur au client.
- Informations retournée par le navigateur (client) lors de chaque interrogation du même serveur HTTP.

- **Catégories** : **sans durée d'expiration** (fichier en mémoire vive)**, avec durée de vie** (fichier sur disque)

- **Ecriture et modification** : fonction **setcookie** dont la syntaxe est la suivante :

  bool **setcookie** (string $name [, string $value [, int $expire = *0* [, string $path [, string $domain [, bool $secure = *false* [, bool $httponly = *false*]]]]]])

- **Lecture** : variable superglogale $_COOKIE

GÉNÉRALITÉS : LES COOKIES

- **Cookies**

- Suppression : Envoie d'un cookie du même nom mais sans valeur.

  setcookie("nom_cookie");

  setcookie ("TestCookie", "", time() - 3600); // précaution sur date de validité

  unset($_COOKIE["nom_cookie"]); // précaution sur l'objet

- Limitations et sécurité

  20 cookies/domaine

  4Ko par cookie

  autres filtres niveau navigateur

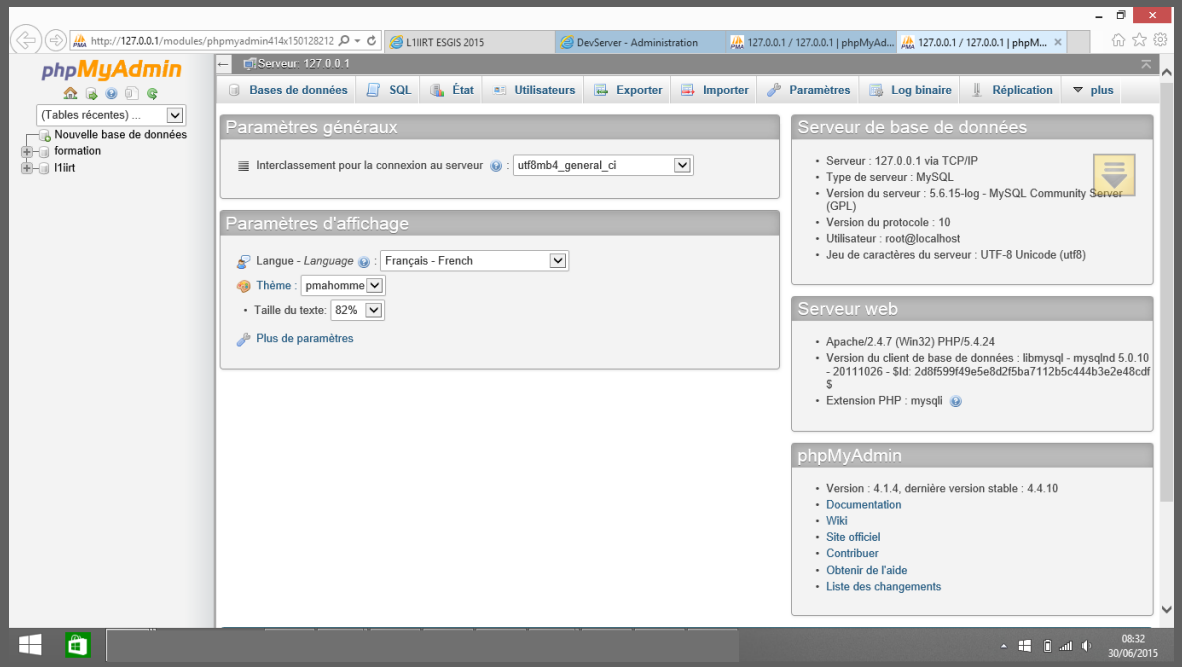
  aucune sécurité (falsification, etc.)

- TP : cas pratique, ***Outil de personnalisation d'affichage***

GÉNÉRALITÉ: PHPMYADMIN

phpMyAdmin est une interface (application) Web développée en PHP et distribuée sous licence GNU GPL pour administrer les bases de données MySQL.

Son interface se présente comme suit :

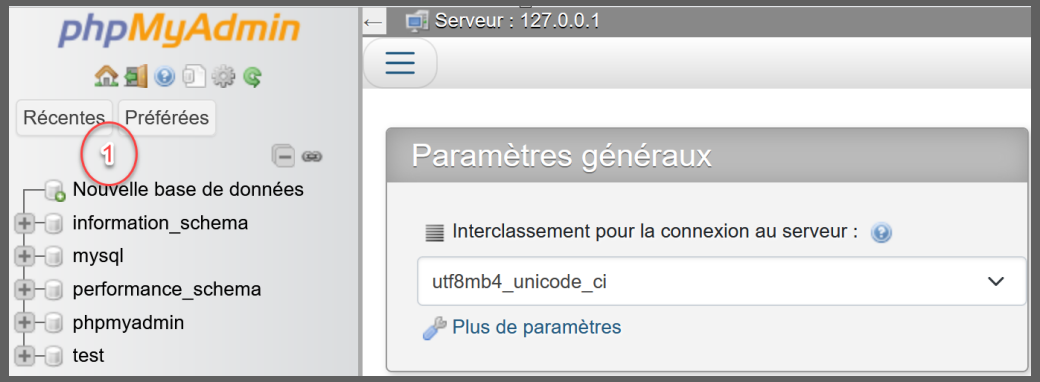


Elle est accessible depuis un navigateur. Mais le chemin d'accès est fonction de l'installation du serveur et|ou de l'hébergement. Il existe souvent des raccourcis via un menu au niveau de la barre des tâches pour la lancer.

GÉNÉRALITÉ: PHPMYADMIN

Notre objectif n'est pas de présenter en détail phpMyAdmin, mais d'en citer quelques fonctionnalités de base.

- **Créer une base de données** : en cliquant sur **« Nouvelle base de données »**



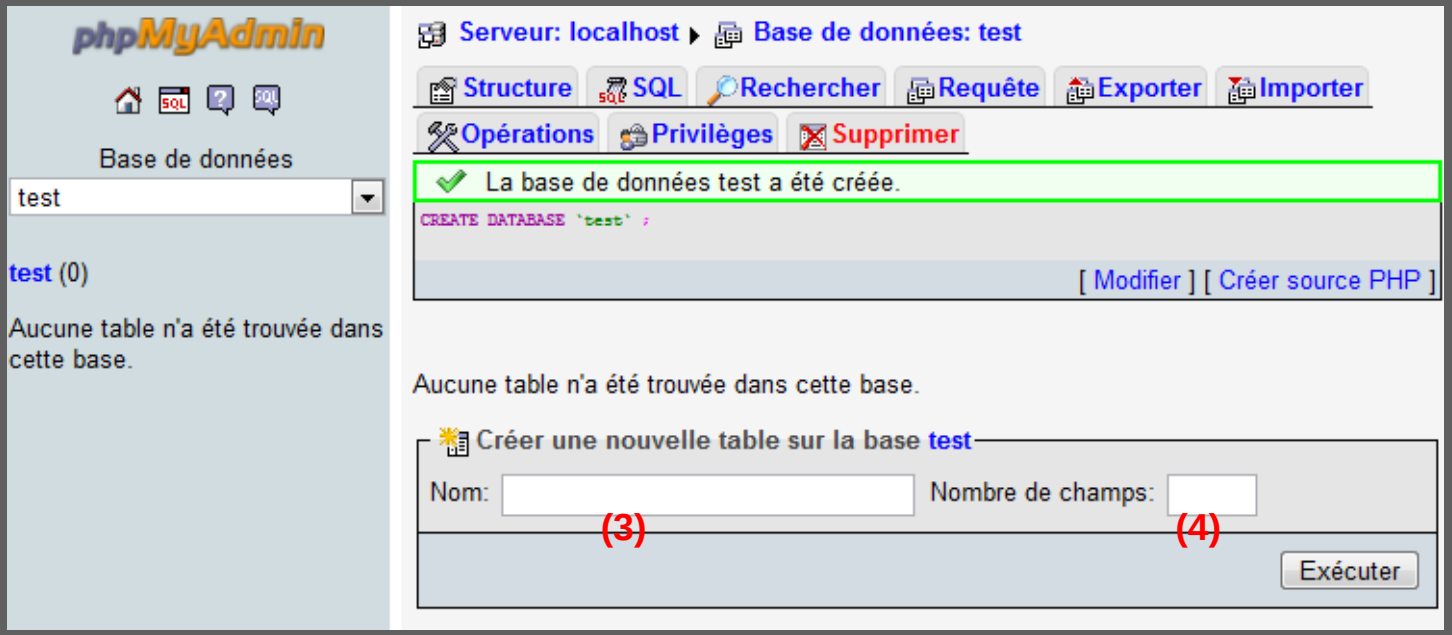


Puis saisir le nom de la base de données dans la zone (2), et cliquer sur le bouton créer pour procéder à la création.

GÉNÉRALITÉ: PHPMYADMIN

- **Créer une table**

A la fin de la création d'une nouvelle base de données, il est automatiquement proposé dans l'interface qui s'affiche, de créer une table. On peut aussi sélectionner, dans la partie gauche, une base de données pour y créer une nouvelle table.

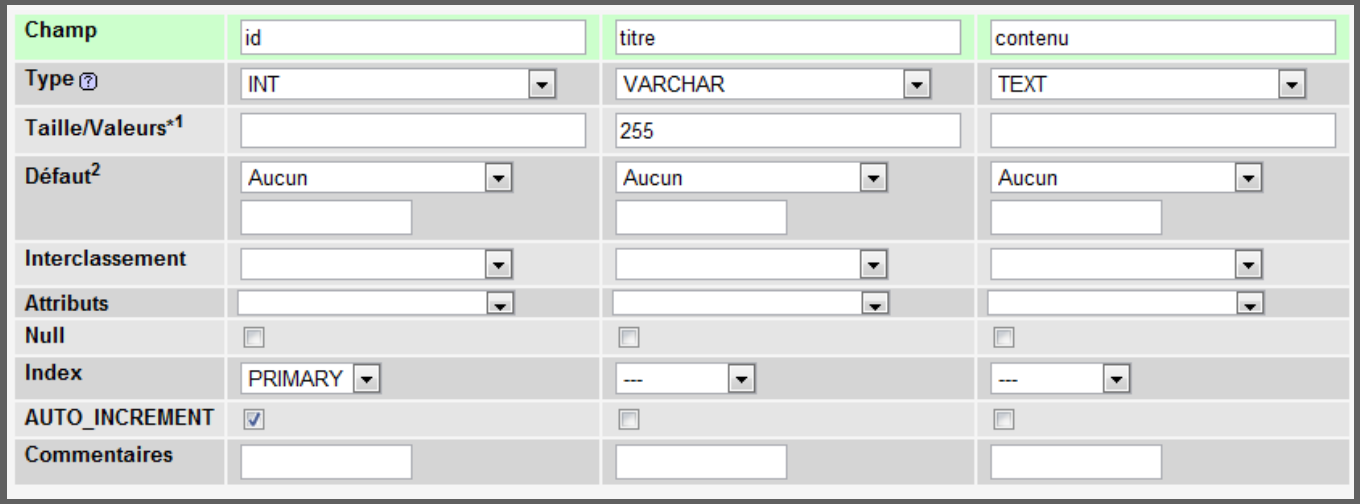


Pour créer une table, il suffit de saisir son nom dans la zone (3) et le nombre de ses champs dans la zone (4) puis cliquer sur le bouton **« Executer »**. Une nouvelle interface va s'afficher pour indiquer la structure de la table (nom des champs ainsi que le type et le format des données qu'ils peuvent contenir.

GÉNÉRALITÉ: PHPMYADMIN

- **Créer une table (suite)**

La première ligne de la figure ci-dessous permet de nommer les différents champs de votre table. Il faut noter que cette ligne peut aussi se présenter sous forme de colonne (verticalement) si la table a beaucoup de champs.



La seconde ligne permet de choisir le type de la donnée contenue dans chaque champ de la table; etc.

Dans l'exemple ci-dessus nous pouvons distinguer les champs « id », « titre », et « contenu » qui sont de type respectifs « INT », « VARCHAR », « TEXT »

Index : active l'indexation du champ. Ici, il s'agit d'une clé primaire « PRIMARY », donc identifiant unique.

AUTO_INCREMENT : permet au champ de s'incrémenter tout seul à chaque ajout d'enregistrement.

A la fin des configurations, il faudra cliquer sur le bouton **« sauvegarder »** pour enfin créer la table. Ce bouton n'est pas visible sur la figure ci-dessus mais il est souvent juste en bas des champs et des paramètres.

GÉNÉRALITÉ: PHPMYADMIN

- **Actions possibles sur une table**

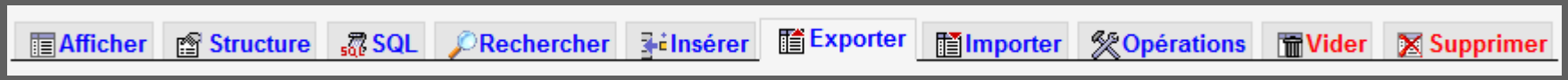
Une fois créée, les actions de bases que l'on peut effectuer sur une table sont les suivantes :

- Insérer une entrée
- Supprimer une ou des entrées
- Modifier une entrée
- Afficher les entrées
- Vider une table
- Supprimer la table
- Effectuer toutes sortes de requêtes autorisées sur la table

- **Actions utiles sur une base de données**

- Sauvegarder (Exporter) une base de données MySQL
- Restaurer (Importer) une base de données MySQL
- Renommer
- Supprimer

Ces différentes actions sont disponibles depuis l'interface graphique de phpMyAdmin suivant que vous sélectionnez à gauche, une base de données ou une table. Il suffit de chercher un tout petit peu en explorant les onglets



PHP ET BASES DE DONNÉES MYSQL

La plupart des opérations réalisées dans l'interface de phpMyAdmin sont directement réalisable en écrivant du code php et en faisant des requêtes. Pour nous simplifier la tâche, nous allons supposer l'existence de bases de données et de tables. Dons, pas de code pour ces deux choses. phpMyAdmin s'en chargera pour nous.

- **Accès à une base de données MySQL depuis PHP**

L'approche recommandée aujourd'hui pour accéder à une base de données reste PDO mais comme à ce niveau de formation, aucune notion d'approche objet n'est encore enseignée, nous allons, pour des raisons pédagogiques, présenter une approche dépréciée mais marche.

Cette approche utilise des fonctions PHP conçues pour attaquer exclusivement des bases de données MySQL.

L'accès à une base de données MySQL se fera généralement en 3 étapes. Une quatrième étape peut cependant exister suivant le type de requête exécutée.

- **Etape 1 : Connexion au serveur hébergeant la base de données**
- **Etape 2 : Sélection de la base de données sur laquelle on veut travailler**
- **Etape 3 : Exécution de la requête voulue**
- **Etape 4 : Exploitation, au besoin, des résultats de la requête**

Chacune de ses étapes est réalisée avec une ou des fonctions PHP dédiées à MyQSL.

Dans le cadre de ce cours nous allons utiliser des fonctions préfixées par **mysqli_**

PHP ET BASES DE DONNÉES MYSQL

- **Etapes 1 et 2 : Connexion au serveur hébergeant la base de données et sélection de cette dernière**

  Il est question d'ouvrir ou de réutiliser une connexion à un serveur MySQL et de sélectionner base de donnée sur ce serveur.

  On va utiliser la fonction **mysqli_connect** dont la syntaxe complète est la suivante :

mysqli mysqli_connect ([string $host = ini_get("mysqli.default_host") [, string $username = ini_get("mysqli.default_user") [, string $passwd = ini_get("mysqli.default_pw") [, string $dbname = "" [, int $port = ini_get("mysqli.default_port") [, string $socket = ini_get("mysqli.default_socket")]]]]])

On y retrouve plusieurs paramètres optionnels dont les plus importants sont :

- Le nom du serveur
- Le nom de l'utilisateur qui se connecte
- Son mot de passe
- Le nom de la base de données

En cas de succès, la fonction retourne **l'identifiant de connexion** MySQL. Dans le cas contraire, elle retourne **FALSE**.

PHP ET BASES DE DONNÉES MYSQL

Exemple de code PHP pour les étapes 1 &2 :

```
$link = mysqli_connect('localhost','root','', 'nom_bdd') or die ('Impossible de se connecter au serveur');
```

- **Etape 3 : Exécution de la requête voulue**

  On utilise le fonction **mysqli_query** dont la syntaxe complète est la suivante :

  mixed mysqli_query (mysqli $link , string $query [, int $resultmode = MYSQLI_STORE_RESULT])

On y retrouve trois paramètres dont 1 est optionnel :

- $query : contient la requête sous forme de chaine de caractères
- $link : représente la ressource (identificateur) caractérisant la connexion MySQL retourné par la fonction mysqli_connect
- $resultmode : Elle a pour valeur soit la constante MYSQLI_USE_RESULT, soit la constante MYSQLI_STORE_RESULT, suivant le comportement désiré. Par défaut, la constante MYSQLI_STORE_RESULT est utilisé..

La valeur retournée par la fonction mysqli_query est fonction du type de requête exécutée.

Pour les requêtes du type SELECT, SHOW, DESCRIBE, EXPLAIN et les autres requêtes retournant un jeu de résultats, la fonction retourne une **ressource en cas de succès**, et **FALSE en cas d'erreur**.

Pour les autres types de requêtes, INSERT, UPDATE, DELETE, DROP, etc., la fonction retourne **TRUE en cas de succès** et **FALSE en cas d'erreur**.

Exemple de code PHP :

```
$result = mysqli_query('SELECT * FROM NOM_DE_VOTRE_TABLE');
```

PHP ET BASES DE DONNÉES MYSQL

- **Etape 4 : Exploitation, au besoin, des résultats de la requête**

  Cette étape n'a de sans que pour les requête de type SELECT.

  On utilise les fonctions préfixées par **mysql_fetch_** et éventuellement une boucle pour parcourir les enregistrements retournés.

mysqli_fetch_xxx (où xxx représente « array », « row », « assoc », « object ») retourne un tableau ou un objet qui contient la ligne demandée dans le résultat **$result** et déplace le pointeur de données interne d'un cran ou pas.

Exemples de code PHP :

```
//pour récupérer le résultat courant
$lign=mysqli_fetch_assoc($result);

//Ou encore pour boucler sur tous les résultats
while($lign=lign=mysqli_fetch_assoc($result) {
   // traiter $lign
}
```

Nota : La page qui suit présente des exemples complets d'utilisation.

PHP ET BASES DE DONNÉES MYSQL

Dans un premier temps on va y utiliser le fichier nommé **connect.inc.php** dont le code est le suivant :

```
<?php
        $host="localhost";
        $login="root";
        $passw="";
        $bdd="votre_bdd";
                $link=mysqli_connect($host,$login,$passw, $bdd) or die("Erreur de connexion au serveur");
?>
```

- **Exemple de sélection d'enregistrement d'une table**

```
<?php
        require_once('connect.inc.php');
        $req="select * from nom_votre_table";
        $result=mysqli_query($link, $req);
        while($lign=mysqli_fetch_assoc($result)) {
                echo $lign["champ1"] ." ". $lign["champ2"] ." ". $lign["champ3"] ... . ."<br/>";
        }
?>
```

- **Exemple d'insertion d'un enregistrement dans une table**

```
<?php
        require_once('connect.inc.php');
        $req="insert into nom_votre_table value('valeur1', 'valeur2',..., 'valeurN')";
        $result=mysqli_query($link, $req);
?>
```

PHP ET BASES DE DONNÉES MYSQL

Quelques fonctions mysqli

mysqli_fetch_row — Retourne une ligne de résultat MySQL sous la forme d'un tableau
mysqli_fetch_array — Retourne une ligne de résultat MySQL sous la forme d'un tableau associatif, d'un tableau indexé, ou les deux
mysqli_fetch_assoc — Lit une ligne de résultat MySQL dans un tableau associatif
mysqli_num_rows — Retourne le nombre de lignes d'un résultat MySQL
mysqli_affected_rows — Retourne le nombre de lignes affectées lors de la dernière opération MySQL

Et beaucoup d'autres ci-dessous dont vous pouvez vérifier la syntaxe d'utilisation sur le site **www.php.net**

mysqli_close
mysql_data_seek
mysqli_errno
mysql_error
mysqli_connect_error
mysqli_escape_string
mysqli_real_escape_string

mysqli_free_result
mysqli_num_rows
mysqli_get_client_info
mysqli_info
mysqli_commit
mysqli_prepare
mysqli_ping

GÉNÉRALITÉS : LES SESSIONS

- **Sessions**

  Réponse aux limitations des cookies => **Stockage coté serveur** des données sensibles des utilisateurs.

  Solution => Sessions

  Objectif => préservations données sensibles contre vol, falsification, etc.

  Principe => - Attribution d'un identifiant de session,
  - Envoie de l'identifiant de session au client,
  - Enregistrement coté serveur (fichier) des informations de session,
  - Enregistrement de l'identifiant au niveau du client (cookie)
  - A chaque requête des pages appropriées du serveur, le navigateur renvoie l'identifiant
  - Le serveur s'en sert pour récupérer dans le fichier correspondant, les informations préalablement stockées

GÉNÉRALITÉS : LES SESSIONS

- **Sessions**

  Initialisation => fonction **session_start(),** placée en haut du script, avant toute sortie vers la page web; comme pour les header (en-tête)

  Si identifiant (provenant du client) alors lecture données stockées sur serveur

  Sinon, création nouveau identifiant, transmission au client (pour stockage), création fichier correspondant (coté serveur)

  Lecture, Ecriture => variable superglobale **$_SESSION[]**

```
// Initialisation de la session
   session_start() ;

// Écriture d'une variable de session
   $_SESSION["nom_variable"] = 'valeur' ;
```

Suppression d'une session => session_destroy() , après initialisation

session_destroy() => suppression du fichier de session (coté serveur) mais pas variables de session.

```
unset($_SESSION);
```

Nota : Des exemples de codes se trouvent sur les deux pages suivantes

GÉNÉRALITÉS : LES SESSIONS

```
<?php   //Code appelé par le formulaire d'authentification
session_start();
if(isset($_SESSION["login"]))  //si une session existait, on la détruit
{
                $_SESSION = array();
                session_destroy();
}
if(isset($_POST["login"]) && isset($_POST["passwd"]) && ! (empty($_POST["login"]) || empty($_POST["login"])))
// si on vient de soumettre un formulaire d'authentification avec tous les champs bien renseignés
{
    require_once("connexion.inc.php"); // on inclut le fichier contenant les paramètres de connexion
                                                    // à la base de données
    $requet=sprintf("SELECT * FROM votre_table_user WHERE login='%s' AND passw='%s'",
                                                mysql_real_escape_string($_POST["login"]),
                                                md5(mysql_real_escape_string($_POST["passwd"])));
   $result=mysqli_query($link,$requet);
   if(mysqli_num_rows($result)==1)  // on doit trouver un seul enregistrement
   {
       $connecte=mysqli_fetch_assoc($result);  //on récupère le seul résultat dans un tableau associatif
                //on enregistre les paramètres de session qu'on souhaite avoir à disposition. Un exemple ci-dessous
                $_SESSION["login"]=$connecte["login"];
                $_SESSION["nom"]=$connecte["prenom"] . '  ' .$connecte["nom"];
                $_SESSION["profil"]=$connecte["profil"];   //peut être utile si on veut gérer les droits des utilisateurs
                                                                //pour décider de quoi afficher
```

GÉNÉRALITÉS : LES SESSIONS

```
                //redirection vers la page souhaitée

                header("Location:votre_page_membre.php");  // c'est dans cette page que vous allez gérer au besoin
                                                                                        // les droits des utilisateurs mais ce n'est pas une
                                                                                // obligation. Tout dépend de ce que vous voulez faire
                }
                else  // En cas de mauvaise identification
                {
                                //redirection vers la page d'authentification
                                header("Location:page_authent.php");
                }
}
else  // En cas de non authentification (non soumission de formulaire d'authentification)
{
                header("Location:page_authent.php");
}

?>
```

GÉNÉRALITÉS : LES SESSIONS

```php
<?php
	//Code appelé pour fermer ou terminer une session
	session_start();
	if(isset($_SESSION["login"]))  //si une session existait, on la détruit
	{
		$_SESSION = array();
		unset($_SESSION);
		session_destroy();
	}
?>

<?php
//Code pour protéger une page par une session
//(elle ne sera accessible qu'aux personnes ayant ouvert une session)
session_start();
if(isset($_SESSION["login"]))  //si une session existe
{
?>
<!- Mettre tout le code (HTML et autres) de votre page ici -->
<?php
}
else
header('Location:page_authent.php');
?>
```